

09 | 集群组件管理：自动化之道，轻松管理百个集群

潘野 · 云原生基础架构实战课



你好，我是潘野。

通过前面的课程，我们学习了通过 GitOps 的方式来管理公有云里的资源，也顺利地启动了一个 Kubernetes 集群。

但是在真正的生产环境中，一个“光秃秃”的集群业务人员是无法使用的。比如业务需要 Ingress 来对外暴露流量，SRE 需要集群内有 Prometheus 来监控整个应用的状态。所以像 Ingress、Prometheus 这些组件依然属于基础设施的管理范畴。也就是说，快速、可靠和可重复的管理 Kubernetes 集群组件也是 IaC 中的一环。

我们不妨试着想象一下，假如你是公司 Kubernetes 平台管理员，需要管理 100 个 Kubernetes 集群，而每个集群都有 10 个组件的时候，你应该如何来做呢？

常见的 Kubernetes 集群组件

我们先看看交付给业务团队使用的 Kubernetes 集群一般需要哪些组件。

这些组件分成两类，一类是控制平面组件，一类是数据平面组件。控制平面的组件里面涉及到网络插件、安全组件等，数据平面的组件主要是以 Ingress、cert-manger 和 Grafana 这样的业务辅助组件为主。我将常用的 Kubernetes 集群组件用表格的形式列在下面，供你参考。

组件名称	用途
Calico/Cilium/Flannel	集群网络组件
Ingress-Controller	七层流量服务组件，常见的有 NGINX Ingress
Istio/Linkerd	服务网格组件
各类SCI Driver，如Rook-ceph	集群存储组件
Rometheus + Alertmanger+ 各种 Exportor	集群监控组件
Grafana / Kibana	logging 或者 metrics 可视化组件
cert-manager	证书管理组件
Velero	集群备份组件
OPA Gatekeeper	集群权限管理组件
KEDA	应用扩缩容组件

极客时间

前面我们预设的场景是 100 个集群，每个集群 10 个组件，那么一共就有 1000 个组件。我们分析一下有哪些通用的管理需求，主要是三类。

- 组件类型管理：**因为 100 个集群可能因为用途不同，就需要针对性的安装不同的组件。例如给 AI 用的 GPU 集群就需要安装 device-plugin 这些组件，而普通集群显然不需要这样的组件。
- 组件版本管理：**这 100 个集群的 Kubernetes 版本可能不一样，那么依赖的组件版本也会不一样。例如 HPA 的 API version 在 1.21 时候是 v2beta1，但是到了 1.24 之后就升级到了 v2，那么在 1.21 集群上涉及到 HPA 的组件可能就不能安装最新版本。
- 组件功能管理：**我们还得考虑不同环境下组件配置不一样的问题。比如在测试环境的集群，我们需要开启组件的一些功能来做测试，而到了正式的生产环境，这些组件中用来测试的功

能就需要关闭。

基于这三类的需要，我们怎么做才能同时实现需求定制，还有集中管理组件这两个要求呢？

如何按需定制集群组件

其实我们仔细想一想，这 100 个 Kubernetes 集群，即便是业务场景不同，但是其中部分集群组件都是一样的。

我们结合自己的经验，很容易就能找到一些共性规律。比如说，集群总需要监控吧？那么就需要采集节点状态 `node-exportor`；再比如，应用总需要对外暴露自己服务的访问地址吧？那么 `ingress-controller` 也是必不可少的，所以这些属于必要组件。

和必要组件相对应，还有一些是可选的功能。比如说，虽然我们要对所有集群进行监控，但是查看监控数据的组件 `Grafana` 并不是每个集群都要安装，只需要安装在某一个集群即可，所以这类组件就属于可选组件。

那么我们是不是可以维护一个集合，来表述一个集群需要安装哪些组件呢？为了帮你理解这个集群组件集合如何实现，我们来看一段非常简单的 `yaml` 配置文件示例。

 复制代码

```
1 ---
2 addons:
3   ingress:
4     enabled: true
5   grafana:
6     enabled: false
7   node-exportor:
8     enabled: true
```

熟悉 `helm` 的同学，看到这里是不是觉得有点眼熟？在 `helm` 的 `values.yaml` 中，我们经常会看到 `enabled: false` 或者是 `enabled: true` 这个字段？既然在 `helm` 中我们可以使用 `enabled` 这个关键词来决定是否开启功能，那我们是不是也可以套用在这里呢？

这肯定是可以的。这时候我们还需要配套使用一个 [🔗技巧](#)，这个技巧来自官方文档。

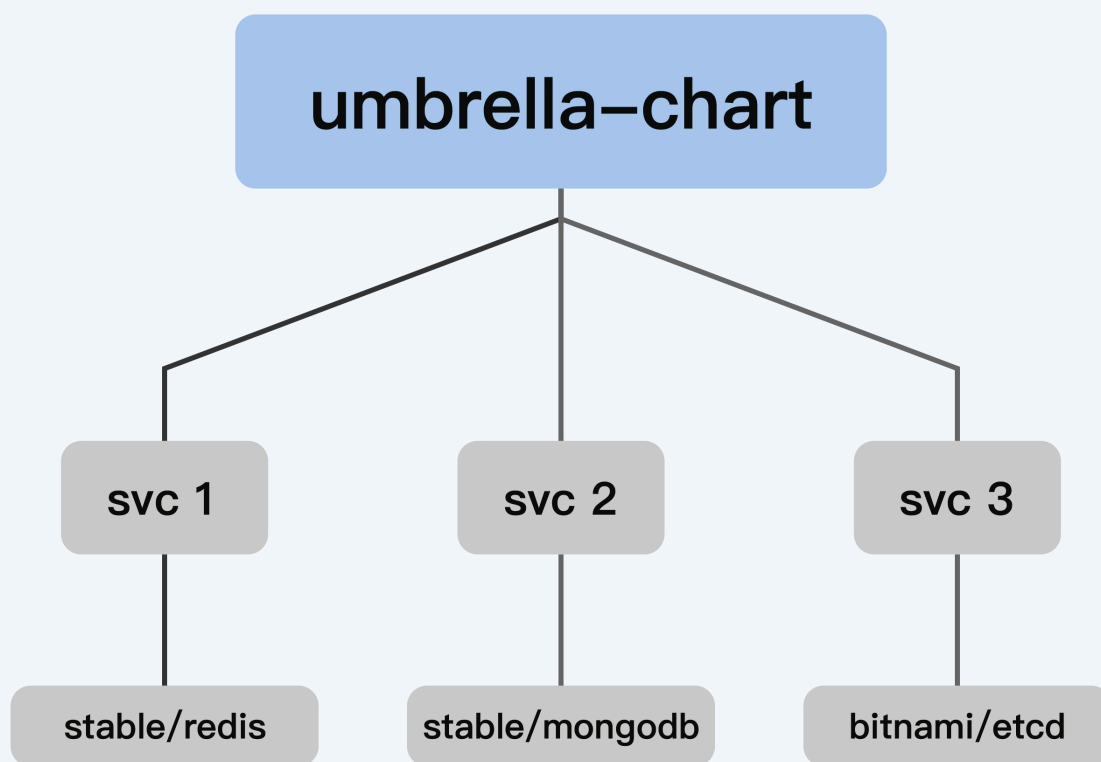
The current best practice for composing a complex application from discrete parts is to create a top-level umbrella chart that exposes the global configurations, and then use the charts/ subdirectory to embed each of the components.

中文翻译过来就是：

构建复杂应用的最佳实践是创建顶层的伞状 Chart (umbrella chart)，用于暴露全局配置，然后使用 charts/ 子目录嵌入每个组件。

那么如何理解顶层的 umbrella chart 呢？

结合我们上面示例的 yaml 配置文件，addons 这个顶层的伞状 chart，在 addons 这个 chart 里，会通过 `enabled` 这个字段来决定是否在集群上启用这些组件。你可以结合下图来理解原理。



那么问题来了，在顶层的 chart 中包含子系统的 chart，这个需求要如何实现呢？

我们需要结合部署集群组件的过程来找到答案。接下来，我们就分三步走，先从部署单个集群组件入手，然后升级到同时部署多个集群组件，最后再进入如何部署多个集群的组件。

ArgoCD：如何部署单个组件

我们一般会通过 Helm 配合 ArgoCD 这个工具来部署**单个**集群组件。这里我以部署 Grafana 为例，快速地讲一下 ArgoCD 部署单个组件的过程，一共分两步。

第一步，在 ArgoCD 中配好 Helm 的仓库。你可以通过 ArgoCD 的 UI 来添加 Helm 仓库，也可以通过命令行来添加仓库。这里的操作也很简单，我们先在 UI 中点击 settings -> Repositories -> connect repo，然后选择 repo 的类型为 helm，再填上 repo 的 URL 即可。

CONNECTSAVE AS CREDENTIALS TEMPLATECANCEL

Choose your connection method:
VIA HTTPS ▼

CONNECT REPO USING HTTPS

Type
helm

Name
grafana

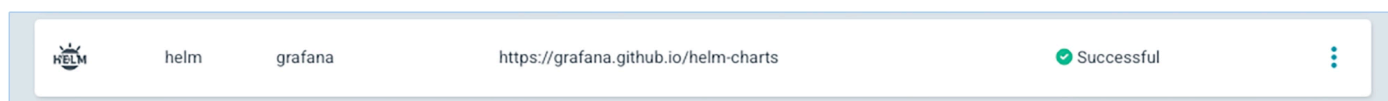
Project
default

Repository URL
https://grafana.github.io/helm-charts

Username (optional)
|



当正确配置了 helm repo 之后，ArgoCD 中就会显示 successful 的提示。

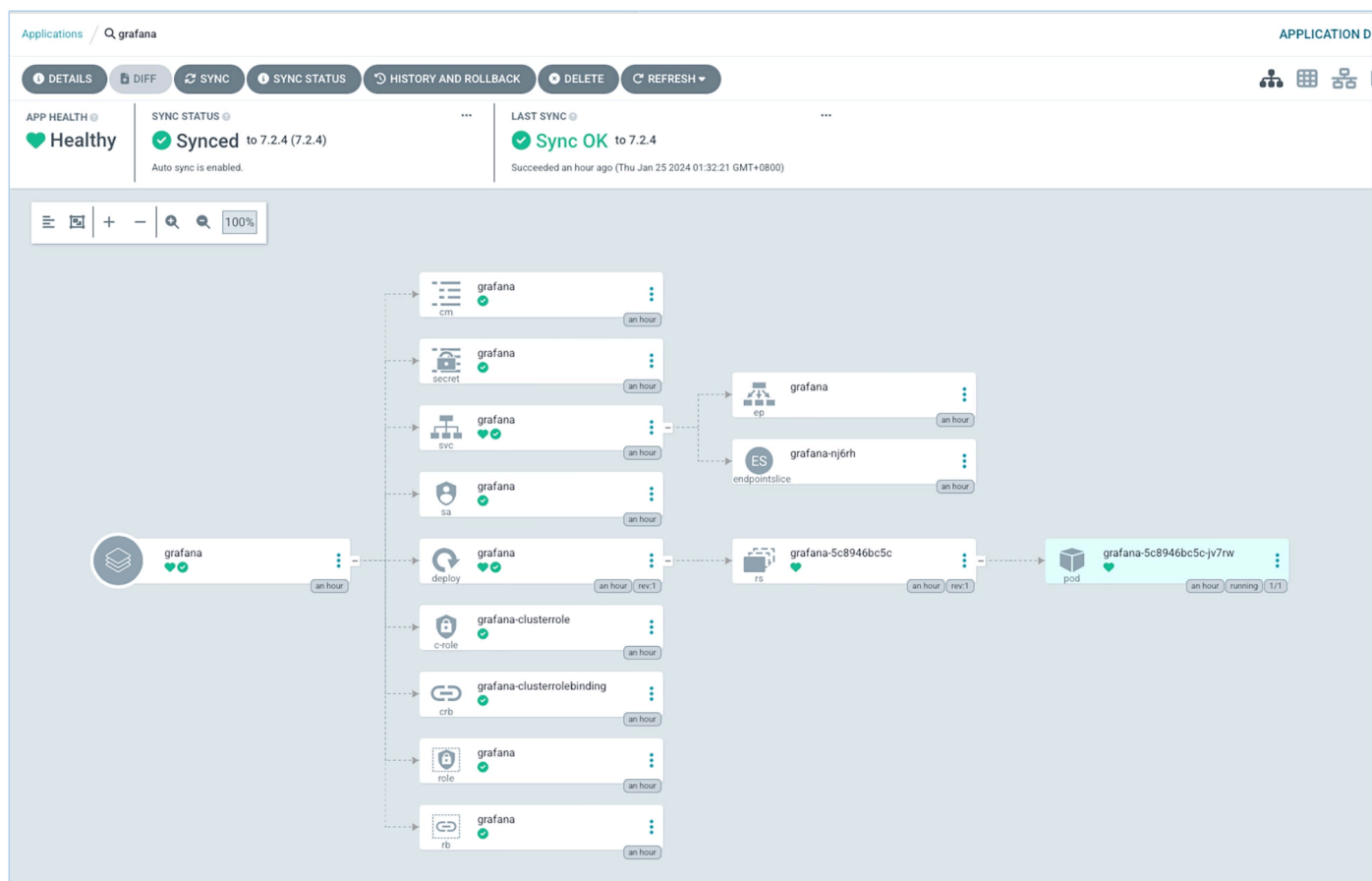


第二步，通过 UI 或者 Kubernetes 命令行创建 ArgoCD 的 Application，配置好相关参数即可部署。

 复制代码

```
1  apiVersion: argoproj.io/v1alpha1
2  kind: Application
3  metadata:
4    name: grafana
5    namespace: argocd
6  spec:
7    destination:
8      namespace: addons-system
9      server: https://Kubernetes.default.svc
10   project: default
11   source:
12     chart: grafana
13     repoURL: https://grafana.github.io/helm-charts
14     targetRevision: 7.2.4
15   syncPolicy:
16     automated: {}
```

完成上面的步骤以后，最终我们就会看到 Grafana 部署在了集群中，如下图所示。



极客时间

umbrella chart – 管理集群所有组件

一个组件是这样部署的，其他组件也同理，也就是说，所有的集群组件都可以通过 Helm 配合 ArgoCD 部署。而 umbrella chart 要做的事情，就是根据 `enabled` 这个字段，渲染出所需要的 Application，这样不就可以解决我们的需求了吗？

我们这就来看看具体实现思路，我们先为 ArgoCD 的 Application 这类资源自定义一个 helm chart 包，将变化的参数转变成可配置的 value。你可以先看看下面的配置示例，再听我分析。

在 template 中 grafana.yaml 中，我们用 `addon.enabled` 来控制 helm 是否渲染出这个配置文件，如果 `enabled` 是 `true`，那么接下来会根据 `values.yaml` 里面的值生成真正的 ArgoCD 的配置文件。下面是 grafana argocd application 的样例代码：

复制代码

```
1 {{- $addon := .Values.addons.grafana }}  
2 {{- if $addon.enabled }}
```

```
3 apiVersion: argoproj.io/v1alpha1
4 kind: Application
5 metadata:
6   name: {{ $addon.names.app }}
7   namespace: argocd
8 spec:
9   destination:
10    namespace: {{ $addon.namespace | default "addons-system" }}
11    server: https://Kubernetes.default.svc
12    project: default
13    source:
14      chart: {{ $addon.names.chart }}
15      helm:
16        valueFiles:
17          - values.yaml
18        repoURL: {{ $addon.repoURL }}
19        targetRevision: {{ $addon.targetVersion }}
20    syncPolicy:
21      automated: {}
22    {{- end }}
```

基于上面的样例代码，我们发现在 helm 中的 value 定义无非就是这么几个参数，分别用来说明是否在集群中启用，chart 的仓库地址，还有需要在集群中安装哪个版本等等信息。你可以参考后面这个例子看一下。

 复制代码

```
1 grafana:
2   enabled: false
3   names:
4     app: grafana
5     chart: grafana
6   repoURL: https://grafana.github.io/helm-charts
7   targetVersion: 7.2.4
```

如何集中化管理集群组件

前面，我们解决了集群组件定制的需求，那么我们如何通过一份代码来管理 100 个集群的组件呢？

Helm 里的 values.yaml 是可以采用层层递进式覆盖的。你可以结合后面这个配置案例来理解。

 复制代码

```
1 apiVersion: argoproj.io/v1alpha1
2 kind: Application
3 metadata:
4   name: addons
5   namespace: argocd
6 spec:
7   destination:
8     namespace: addons-system
9     server: https://Kubernetes.default.svc
10  project: default
11  source:
12    chart: addonsmgr
13    helm:
14      valueFiles:
15        - values.yaml
16        - values/aks/infra.yaml
17    repoURL: https://cloudnative-automation.github.io/addon-manager/repo
18    targetRevision: 0.1.*
19  syncPolicy:
20    automated: {}
```

可以看到，我们只要为每个集群单独配置一个 values.yaml，就能控制集群可以安装哪些组件。这是因为 valueFiles 会默认读取 chart 包中的 values.yaml，并覆盖掉默认的 values。

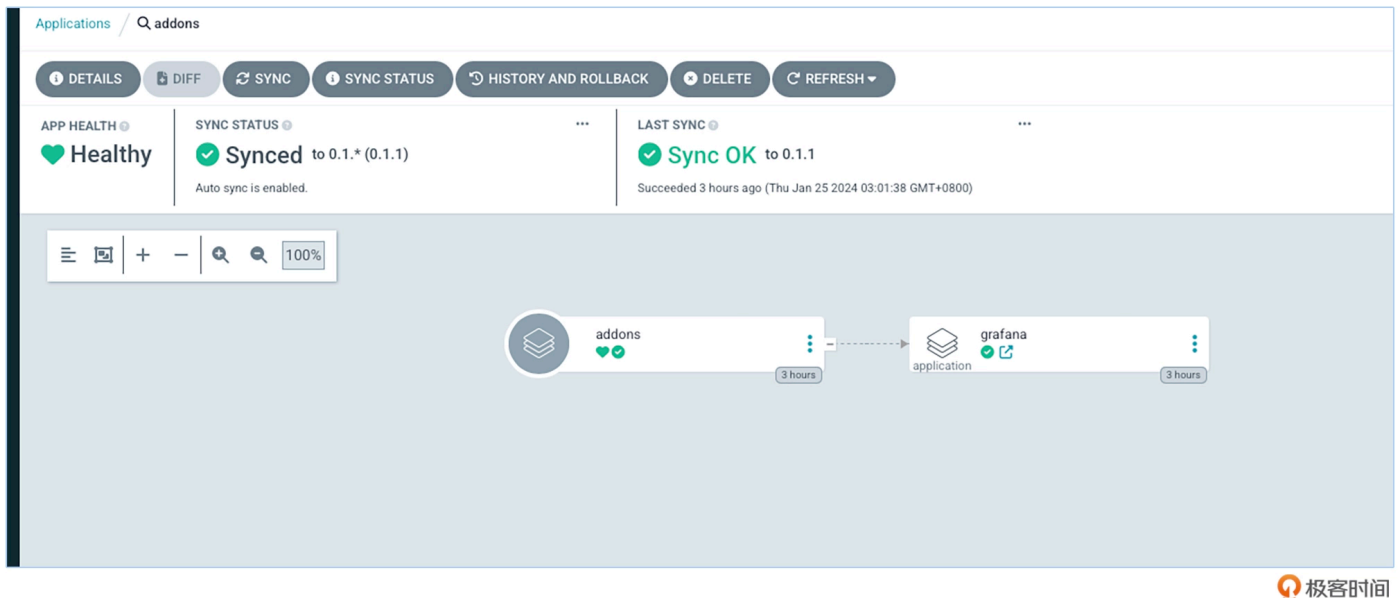
例如，我们在 `values/aks/infra.yaml` 中定义了 Grafana 开启，而 ingressNginx 则没有开启，这样就能精细地控制这个集群可以安装哪些组件。

 复制代码

```
1 ---
2 clusterURL: infra-dev.hcp.japaneast.azmk8s.io
3
4 addons:
5
6   grafana:
7     enabled: true
8
9   ingressNginx:
```

效果图

让我们来看下这样做的效果如何。



对照截图可以看到，addons 是顶层的 umbrella chart，在这个集群中，Grafana 是它的子 chart。详细代码我已经放在 [GitHub](#) 上，你可以结合我前面的讲述来深入理解。

看到这里，有同学可能会问，想要建立 100 个集群，岂不是要执行 100 次创建命令了么？

这一点其实你无需担心。在工作中，我们同时建立的集群一般不超过 10 个。我们只是在集群初始化时，才会去创建这个 addons 的 application，部署 addons 这个顶层 chart。之后，ArgoCD 将会通过 `targetRevision: 0.1.*` 这样的方式来监听 Helm 仓库的变化，一旦有新的版本出现，便会自己同步过来，进行更新。

如何维护 helm chart 的仓库地址

除了创建命令执行的自动化，对于 helm chart 的仓库地址的维护，我们也可以自动化实现。

我们可以看到，每个子 chart 都有一个 repoURL 的配置，常用的管理方式有两种方式。

第一种是自己维护一个 chart repo，将所有需要安装的 chart 包导入自己维护的 repo 中，同学们可以参考课程 [🔗GitHub 仓库](#) 中关于自建 chart 仓库的说明。

第二种是依然使用原始 chart 的仓库地址，这里需要保证集群环境网络对外的连通性。

 复制代码

```
1 source:
2   chart: grafana
3   repoURL: https://grafana.github.io/helm-charts
4   targetRevision: 7.2.4
```

无论是哪一种管理方式，都需要提前将 Chart 的 Repositories 导入 ArgoCD。这一讲里我已经写了如何在 ArgoCD 中导入仓库的方法，这步动作也是在集群初始化的时候先完成，你可以把后面这段代码放在 Terraform 的代码中。

 复制代码

```
1 resource "repo_resource" "add helm repos" {
2
3   provisioner "local-exec" {
4     command = <<EOF
5       argocd repo add https://charts.jetstack.io --username devops;
6     EOF
7
8     environment = {
9       AWS_REGION = var.region
10    }
11  }
```

GitOps 维护

讲到这里，我们已经完成了所有的工作。

如果我们想在某个集群里打开一个组件，应该如何操作呢？

例如我们在集群中打开 cert- manager 这个组件，只需要提交一个 PR，经过代码的审核，然后 merge 进代码主干。

enable cert manager in aks cluster #1

Open

pye99 wants to merge 1 commit into `main` from `enable_certmanager`

Conversation 0

Commits 1

Checks 0

Files changed 1

Changes from all commits ▾ File filter ▾ Conversations ▾ Jump to ▾ ⚙ ▾

5 addons/values/aks/infra.yaml

↑...	@@ -7,4 +7,7 @@ addons:
7	7 enabled: true
8	8
9	9 ingressNginx:
10	- enabled: false
	⊖
	+ enabled: false
11	+
12	+ certManager:
13	+ enabled: true

这样就能触发 GitHub Action 的流水线，将我们的 Chart 自动打包并上传到镜像仓库，ArgoCD 监测到 chart 版本变化之后开始更改集群组件配置。

All checks have passed

3 successful checks

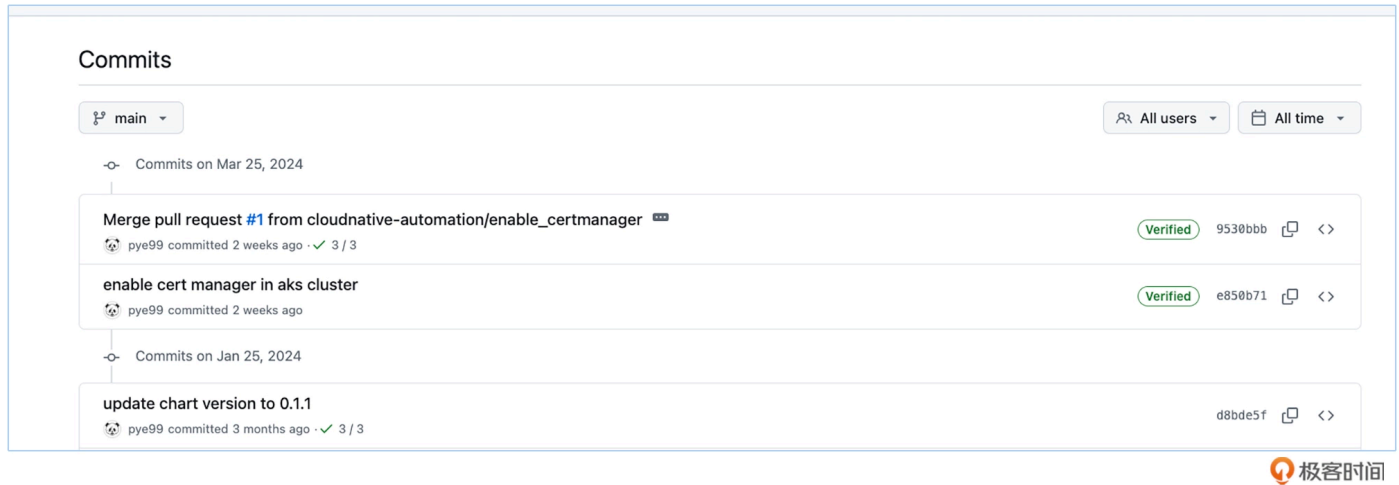
pages build and deployment / build (dynamic) Successful in 20s [Details](#)

pages build and deployment / report-build-status (dynamic) Successful in 3s [Details](#)

pages build and deployment / deploy (dynamic) Successful in 9s [Details](#)

如果我们需要回滚某个配置，只需要重新提交一个 git revert 的 PR，之后 GitHub Action 的流水线以及 ArgoCD 会遵循同样的方法部署出去。

进行到这里，我们实践了如何用 GitOps 方式来管理 Kubernetes 集群配置。在这个 [仓库](#) 里，你可以看到整个集群的配置更改历史，都显示在 Git 的 commit 中，而且使用了 Git 作为唯一真实数据源的运维模式。



这种方式提供了版本控制、审计日志以及回滚等功能，完整地实现了我们在课程最初提出来的快速、可靠和可重复的部署和管理过程。

总结

今天这一讲是基础架构自动化管理中相当重要的一节课。

我们的目标是让 Kubernetes 基础组件既可以按需求定制，又能够集中管理。为此，我们从我们 umbrella chart 切入为你做了详细演示。

我们先从部署单个集群组件入手，然后升级到同时部署多个集群组件，最后再进入如何部署多个集群的组件。

由此，我们就解决了基础设施自动化管理的最后一公里，也就是 Kubernetes 集群交付。同时也遵循了我们在课程最初提出的快速、可靠和可重复的部署和管理过程。

到今天为止，我们完成了基础设施即代码环节，但是目前我们仅仅完成了交付给业务使用的环节。在此基础上，我们如何保证交付的基础设施的可用性与稳定性呢？这些问题后面我们再继续讨论，敬请期待。

思考题

请你根据 [🔗 代码仓库](#) 里提供的样例，来完成其他集群组件的部署。

 复制代码

```
1 ---
2 clusterURL: 127.0.0.1
3
4 addons:
5
6   namespaces:
7     default: addons-system
8
9   grafana:
10     enabled: false
11     names:
12       app: grafana
13       chart: grafana
14     repoURL: https://grafana.github.io/helm-charts
15     targetVersion: 7.2.4
16
17   ingressNginx:
18     enabled: false
19     names:
20       app: ingress-nginx
21       chart: ingress-nginx/ingress-nginx
22     repoURL: https://kubernetes.github.io/ingress-nginx
23     targetVersion: 4.9.0
```

欢迎在评论区与我讨论。如果这一讲对你有启发，也欢迎分享给身边更多朋友。

AI智能总结

1. 管理Kubernetes集群组件是必不可少的，需要考虑组件类型管理、版本管理和功能管理等需求。
2. 通过维护一个集合来表述一个集群需要安装哪些组件，可以实现需求定制和集中管理组件。
3. 使用umbrella chart结合enabled字段来决定是否在集群上启用组件，实现按需定制集群组件。
4. Helm配合ArgoCD工具可以部署单个集群组件，同时umbrella chart可以管理集群所有组件。
5. 通过为ArgoCD的Application自定义一个helm chart包，将变化的参数转变成可配置的value，可以实现集中

化管理集群组件。

6. Helm的values.yaml可以采用层层递进式覆盖，为每个集群单独配置一个values.yaml，就能控制集群可以安装哪些组件。

7. GitOps方式可以用来管理Kubernetes集群配置，提供版本控制、审计日志以及回滚等功能。

8. 通过GitOps方式管理Kubernetes集群配置，可以实现快速、可靠和可重复的部署和管理过程。

9. 基础设施自动化管理的目标是让Kubernetes基础组件既可以按需求定制，又能够集中管理。

10. 本文介绍了如何部署单个集群组件、同时部署多个集群组件，以及如何部署多个集群的组件，解决了基础设施自动化管理的最后一公里。

© 版权归极客邦科技所有，未经许可不得传播售卖。页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

全部留言 (5)

最新 精选



cfanbo

2024-04-15 来自浙江

上一节介绍了tekton，为什么这节才用了argoCD，是因为什么原因？生态？

作者回复: 其实两个不是一个类型的工具。

Tekton注重的是流水线，持续集成的过程。而ArgoCD关注点是自动化部署。



1



cfanbo

2024-04-15 来自浙江

它与argoCD比较有哪些优缺点？

作者回复: 我想你可能想问的是argocd workflow。区别在流程式和竞争式的区别，workflow注重的是前一步的输出是下一步的输入，而竞争式是靠不断的轮训最终达到一致性。



Geek_45a572

2024-04-15 来自陕西

老师您好，如何完整的拉起一套系统呢？像云服务的rds等其他saas服务如何集成到terraform？数据库的参数是否需要terraform管理呢？

作者回复: RDS, VPC这些都可以集成在terraform中，根据你的需要，把相关资源与应用绑定，或者与应用解耦，我们内部项目里，几乎做到了从VPC到安全策略，从Kubernetes到RDS，从ELB到S3，都是用terraform管理起来的。但是这里的确是需要具体业务形态具体分析，没有什么通用范式。



咸蛋超人

2024-04-12 来自广东

潘老师，能否分享一下再eBay这种公司是怎么样来管理和运维这么多的大规模K8S集群？是否有文章或者资料可以分享下？

作者回复: Hi 咸蛋超人

这个视频可供参考：

https://www.youtube.com/watch?v=_uxeum9-TPE



kissyouddyb

2024-08-07 来自广东

太有用了，舒服，真是开了眼界

